

SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – APPLICATION BASED QUESTIONS

Course Code:	DSA0501
Course Title:	QUERY PROCESSING
CO Assessed:	CO1
Assessment:	Tool 2 – Application Based Questions
Weightage:	_____
Total Marks:	20 Marks

Student Name: SHABARISH N Reg. No.: 192424007

Section/Batch: A SLOT & 3RD YEAR Date of Submission: 27-

07-2026 Faculty In-charge: Dr. VEENA DEVI K Submission

Mode: Individual

Code of Conduct

I SHABARISH N 19242007 certify that this submission is my original work and have adhered to all academic integrity guidelines.

Criteria	Marks
Understanding of the Problem	4
Application of Concepts	4
Program Design / Algorithm	4
Correctness of Solution & Output	4
Explanation / Interpretation of Results	4
Total	20

CO1 – AT2 – Application Based Questions

Question 1: University Placement Records (10 Marks)

Problem Statement: Student details are available in a CSV file, while placement information is stored in JSON format. Write a Python program to integrate the datasets, identify students who have received placement offers, and prepare a placement summary.

1.1 Approach

- Read student master data (StudentID, Name, Department, CGPA) from a CSV file using the csv module.
- Read placement data (StudentID, Company, Package, Status) from a JSON file using the json module.
- Integrate (merge) both datasets on the common key StudentID.
- Filter students whose Status is 'Offered' to identify placed students.
- Compute summary statistics: total students, total placed, placement percentage, average package, highest package, and department-wise placement counts.
- Display a formatted placement summary report.

1.2 Sample Input Data

students.csv:

```
StudentID,Name,Department,CGPA
S101,Aarav Sharma,Computer Science,8.9
S102,Diya Patel,Electronics,7.8
S103,Kabir Singh,Computer Science,6.5
S104,Meera Nair,Mechanical,7.2
S105,Rohan Gupta,Information Technology,9.1
S106,Sneha Reddy,Electronics,6.9
S107,Vikram Rao,Computer Science,8.2
S108,Anjali Iyer,Mechanical,7.5
```

placements.json:

```
[
  {"StudentID": "S101", "Company": "TCS", "Package_LPA": 7.5, "Status": "Offered"},
  {"StudentID": "S103", "Company": "Infosys", "Package_LPA": 4.5, "Status": "Not Offered"},
  {"StudentID": "S105", "Company": "Google", "Package_LPA": 22.0, "Status": "Offered"},
  {"StudentID": "S106", "Company": "Wipro", "Package_LPA": 4.0, "Status": "Not Offered"},
  {"StudentID": "S107", "Company": "Amazon", "Package_LPA": 18.5, "Status": "Offered"},
  {"StudentID": "S108", "Company": "Tech Mahindra", "Package_LPA": 5.0, "Status": "Offered"}
]
```

1.3 Python Program

```
"""
University Placement Records
-----
Integrates student details (CSV) with placement information (JSON),
identifies students who received placement offers, and prepares a
placement summary report.
"""

import csv
import json
```

```

def read_students(csv_file):
    """Read student details from a CSV file into a dictionary keyed by StudentID."""
    students = {}
    with open(csv_file, newline="", encoding="utf-8") as f:
        reader = csv.DictReader(f)
        for row in reader:
            students[row["StudentID"]] = {
                "Name": row["Name"],
                "Department": row["Department"],
                "CGPA": float(row["CGPA"]),
            }
    return students

def read_placements(json_file):
    """Read placement records from a JSON file into a dictionary keyed by StudentID."""
    with open(json_file, encoding="utf-8") as f:
        data = json.load(f)

    placements = {}
    for record in data:
        placements[record["StudentID"]] = {
            "Company": record["Company"],
            "Package_LPA": record["Package_LPA"],
            "Status": record["Status"],
        }
    return placements

def integrate_data(students, placements):
    """Merge student details with placement details into a single dataset."""
    combined = []
    for sid, sinfo in students.items():
        record = {
            "StudentID": sid,
            "Name": sinfo["Name"],
            "Department": sinfo["Department"],
            "CGPA": sinfo["CGPA"],
            "Company": placements.get(sid, {}).get("Company", "Not Placed"),
            "Package_LPA": placements.get(sid, {}).get("Package_LPA", 0),
            "Status": placements.get(sid, {}).get("Status", "Not Offered"),
        }
        combined.append(record)
    return combined

def get_placed_students(combined):
    """Filter students who have received a placement offer."""
    return [rec for rec in combined if rec["Status"] == "Offered"]

def generate_summary(combined, placed_students):
    """Build a placement summary from the integrated dataset."""
    total_students = len(combined)
    total_placed = len(placed_students)
    placement_pct = (total_placed / total_students * 100) if total_students else 0

    dept_wise = {}

```

```

for rec in placed_students:
    dept_wise[rec["Department"]] = dept_wise.get(rec["Department"], 0) + 1

avg_package = (
    sum(rec["Package_LPA"] for rec in placed_students) / total_placed
    if total_placed else 0
)
highest = max(placed_students, key=lambda r: r["Package_LPA"], default=None)

return {
    "total_students": total_students,
    "total_placed": total_placed,
    "placement_percentage": round(placement_pct, 2),
    "department_wise_placements": dept_wise,
    "average_package_lpa": round(avg_package, 2),
    "highest_package_record": highest,
}

def print_report(combined, placed_students, summary):
    print("=" * 70)
    print("PLACEMENT SUMMARY REPORT".center(70))
    print("=" * 70)

    print("\nStudents who received placement offers:\n")
    header = f"{'ID':<6}{ 'Name':<16}{ 'Department':<24}{ 'Company':<16}{ 'Package(LPA)':<12}"
    print(header)
    print("-" * len(header))
    for rec in placed_students:
        print(f"{'StudentID':<6}{rec['Name']:<16}{rec['Department']:<24}"
              f"{'Company':<16}{rec['Package_LPA']:<12}")

    print("\n" + "-" * 70)
    print(f"Total Students           : {summary['total_students']}")
    print(f"Total Placed                   : {summary['total_placed']}")
    print(f"Placement Percentage          : {summary['placement_percentage']}%")
    print(f"Average Package (LPA)        : {summary['average_package_lpa']}")
    if summary["highest_package_record"]:
        h = summary["highest_package_record"]
        print(f"Highest Package           : {h['Package_LPA']} LPA ({h['Name']} - {h['Company']})")

    print("\nDepartment-wise placement count:")
    for dept, count in summary["department_wise_placements"].items():
        print(f"    {dept:<24}: {count}")
    print("=" * 70)

def main():
    students = read_students("students.csv")
    placements = read_placements("placements.json")

    combined = integrate_data(students, placements)
    placed_students = get_placed_students(combined)
    summary = generate_summary(combined, placed_students)

    print_report(combined, placed_students, summary)

if __name__ == "__main__":

```

```
main()
```

1.4 Sample Output

```
=====
                        PLACEMENT SUMMARY REPORT
=====

Students who received placement offers:

ID      Name          Department          Company          Package(LPA)
-----
S101    Aarav Sharma        Computer Science    TCS              7.5
S105    Rohan Gupta          Information Technology Google           22.0
S107    Vikram Rao           Computer Science    Amazon           18.5
S108    Anjali Iyer          Mechanical          Tech Mahindra     5.0
-----

Total Students      : 8
Total Placed        : 4
Placement Percentage : 50.0%
Average Package (LPA) : 13.25
Highest Package     : 22.0 LPA (Rohan Gupta - Google)

Department-wise placement count:
  Computer Science   : 2
  Information Technology : 1
  Mechanical         : 1
=====
```

Question 2: Public Library Membership Analysis (10 Marks)

Problem Statement: A library stores member details in CSV format and book borrowing records in XML format. Write a Python program to identify members who have borrowed more than five books and generate a report containing their details.

2.1 Approach

- Read member master data (MemberID, Name, Email, JoinYear) from a CSV file using the csv module.
- Parse book borrowing records from an XML file using the xml.etree.ElementTree module.
- Count the number of books borrowed by each member using their MemberID.
- Identify members whose borrow count is greater than five (the threshold).
- Combine each frequent borrower's member details with their borrowing count and list of borrowed titles.
- Display a formatted report of frequent borrowers, sorted by number of books borrowed.

2.2 Sample Input Data

members.csv:

```
MemberID,Name,Email,JoinYear
M001,Arjun Mehta,arjun.mehta@example.com,2021
M002,Priya Menon,priya.menon@example.com,2022
M003,Karan Malhotra,karan.malhotra@example.com,2020
M004,Ishita Desai,ishita.desai@example.com,2023
M005,Farhan Khan,farhan.khan@example.com,2019
```

borrow_records.xml (excerpt):

```
<Record>
  <MemberID>M001</MemberID>
  <BookTitle>Introduction to Algorithms</BookTitle>
  <BorrowDate>2025-01-10</BorrowDate>
</Record>
<Record>
  <MemberID>M001</MemberID>
  <BookTitle>Clean Code</BookTitle>
  <BorrowDate>2025-01-20</BorrowDate>
</Record>
... (additional <Record> elements follow the same structure)
```

2.3 Python Program

```
"""
Public Library Membership Analysis
-----
Integrates member details (CSV) with book borrowing records (XML),
identifies members who have borrowed more than five books, and
generates a report containing their details.
"""

import csv
import xml.etree.ElementTree as ET
from collections import defaultdict

BORROW_THRESHOLD = 5 # "more than five books"

def read_members(csv_file):
    """Read member details from a CSV file into a dictionary keyed by MemberID."""
    members = {}
    with open(csv_file, newline="", encoding="utf-8") as f:
        reader = csv.DictReader(f)
        for row in reader:
            members[row["MemberID"]] = {
                "Name": row["Name"],
                "Email": row["Email"],
                "JoinYear": row["JoinYear"],
            }
    return members

def read_borrow_records(xml_file):
    """Parse the XML file and count books borrowed per member."""
    tree = ET.parse(xml_file)
    root = tree.getroot()

    borrow_counts = defaultdict(int)
    borrowed_titles = defaultdict(list)

    for record in root.findall("Record"):
        member_id = record.find("MemberID").text.strip()
        title = record.find("BookTitle").text.strip()
        borrow_counts[member_id] += 1
        borrowed_titles[member_id].append(title)
```

```

        return borrow_counts, borrowed_titles

def find_frequent_borrowers(members, borrow_counts, borrowed_titles,
threshold=BORROW_THRESHOLD):
    """Identify members who have borrowed more than `threshold` books."""
    frequent_borrowers = []
    for member_id, count in borrow_counts.items():
        if count > threshold:
            member_info = members.get(member_id, {})
            frequent_borrowers.append({
                "MemberID": member_id,
                "Name": member_info.get("Name", "Unknown"),
                "Email": member_info.get("Email", "Unknown"),
                "JoinYear": member_info.get("JoinYear", "Unknown"),
                "BooksBorrowed": count,
                "Titles": borrowed_titles[member_id],
            })
    frequent_borrowers.sort(key=lambda r: r["BooksBorrowed"], reverse=True)
    return frequent_borrowers

def print_report(frequent_borrowers):
    print("=" * 70)
    print("LIBRARY MEMBERSHIP ANALYSIS REPORT".center(70))
    print(f"Members who borrowed more than {BORROW_THRESHOLD} books".center(70))
    print("=" * 70)

    if not frequent_borrowers:
        print("\nNo members exceeded the borrowing threshold.")
        return

    for rec in frequent_borrowers:
        print(f"\nMember ID      : {rec['MemberID']}")
        print(f"Name                : {rec['Name']}")
        print(f"Email               : {rec['Email']}")
        print(f"Join Year           : {rec['JoinYear']}")
        print(f"Books Borrowed: {rec['BooksBorrowed']}")
        print("Book Titles      :")
        for i, title in enumerate(rec["Titles"], start=1):
            print(f"    {i}. {title}")
        print("-" * 70)

    print(f"\nTotal frequent borrowers found: {len(frequent_borrowers)}")
    print("=" * 70)

def main():
    members = read_members("members.csv")
    borrow_counts, borrowed_titles = read_borrow_records("borrow_records.xml")
    frequent_borrowers = find_frequent_borrowers(members, borrow_counts, borrowed_titles)
    print_report(frequent_borrowers)

if __name__ == "__main__":
    main()

```

2.4 Sample Output

```
=====
```

```
LIBRARY MEMBERSHIP ANALYSIS REPORT
Members who borrowed more than 5 books
=====

Member ID      : M001
Name           : Arjun Mehta
Email          : arjun.mehta@example.com
Join Year      : 2021
Books Borrowed: 6
Book Titles    :
    1. Introduction to Algorithms
    2. Clean Code
    3. The Pragmatic Programmer
    4. Design Patterns
    5. Refactoring
    6. Head First Java
-----

Member ID      : M003
Name           : Karan Malhotra
Email          : karan.malhotra@example.com
Join Year      : 2020
Books Borrowed: 6
Book Titles    :
    1. The Alchemist
    2. Ikigai
    3. Deep Work
    4. Thinking, Fast and Slow
    5. The Power of Habit
    6. Grit
-----

Total frequent borrowers found: 2
=====
```

2.5 Result Summary Table

Member ID	Name	Books Borrowed
M001	Arjun Mehta	6
M003	Karan Malhotra	6